



Knowledge-based job scheduling for HPC

Guilherme Diel¹ · Ana Eloina Nascimento Kraus¹ · Guilherme Piêgas Koslovski^{1,2}

Received: 4 March 2024 / Revised: 3 June 2024 / Accepted: 18 August 2024 / Published online: 21 January 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

Scheduling jobs for executing atop large-scale High-Performance Computing (HPC) infrastructures is a fundamental administrative challenge essential to maximize the productivity indicators of such complex Data Centers (DCs). Several policies were proposed by specialized literature, considering specific scenarios and applications, or proposing DC-tailored heuristics. However, the definition of the appropriate policy depends directly on the applications and infrastructure load, that is, indicators that vary over time. Given this context, this work advances the field on applying Machine Learning (ML) techniques to improve the existing scheduling policies. Specifically, we propose Knowledge-based Job Scheduler (KJS), which uses regression techniques to characterize the performance indicators of existing schedulers, composing a knowledge database, and instead of proposing a one-size-fits-all scheduling function, KJS consolidates the information and uses data classification for defining the jobs' execution order. The simulation campaign demonstrates that KJS can adapt to different workloads when compared to the individual use of existing policies. Essentially, KJS combines polynomial data regression and classification for improving the performance indicators of HPC DC, in both the users' and the administrators' perspectives.

Keywords Scheduling · Supervised learning · Linear regression · High performance computing

1 Introduction

High-Performance Computing (HPC) is a supporting pillar for the development of new technologies in multiple research fields. Applications come from different areas for improving research and development skills on the use of wind farms, quantum chemistry, machine learning, among others. In parallel, powerful HPC Data Centers (DCs) are constantly researched, elaborated and deployed [1, 2], resulting on large-scale infrastructures with high

administrative and operational costs. In this sense, the management of an HPC DC constitutes a critical administrative task, specifically regarding the resources and job scheduling. Applications are submitted to Resources and Jobs Management Systems (RJMSs), and internally represented as jobs. Each job is composed of tasks, which represent the individual processing units, and multiple jobs can share an underlying HPC infrastructure.

Scheduling algorithms are constantly researched and analyzed to improve the overall HPC DC and user indicators. Given the particularities and challenges introduced by heterogenous jobs and DC load, Machine Learning (ML)-based techniques are largely applied to dynamically adapt the scheduling policy to workload and DC status. In fact, ML is used mainly in two scheduling-related tasks: the creation of new queueing policies and to improve the state-of-the-art policies tailoring the performance to specific workloads. Although both research lines proved the importance of ML for scheduling jobs, recent works claim that the heuristic-generated knowledge composed a powerful dataset, that should not be ignored [3, 4].

Recently, some works paved the road towards composing large datasets with scheduling decisions, and later

✉ Guilherme Piêgas Koslovski
guilherme.koslovski@udesc.br

Guilherme Diel
guilherme.diel@edu.udesc.br

Ana Eloina Nascimento Kraus
aen.kraus0010@edu.udesc.br

¹ Laboratory of Parallel and Distributed Processing,
Department of Computer Science, Santa Catarina State
University, Joinville, SC, Brazil

² Graduate Program in Applied Computing, Santa Catarina
State University, Joinville, SC, Brazil

improving the overall performance by extracting qualitative information from them [5–10]. Following this line, we proposed an offline trained scheduler, based on polynomial regression and data classification, to improve the HPC job scheduling considering the perspectives of users and HPC DC administrators. Our approach is based on four phases: (i) a representative dataset is composed based on well-known policies and publicly available HPC execution traces; (ii) each decision performed by the tutors (pre-selected policies from specialized literature) receives three scores, representing the DC and the users' objectives; (iii) polynomial regression is used to predict scores for unknown workloads and DCs status; and (iv) data classification is used to sort out the combination of the original dataset and the predicted scores.

Our Knowledge-based Job Scheduler (KJS) largely uses data regression and classification for defining the appropriate task execution ordering. While the use of different approaches for data regression and task scoring were previously investigated and consolidated [5], we deepen the discussion by jointly applying data classification for defining the execution priorities. Instead of proposing a new one-size-fits-all scheduling function, KJS uses data regression for identifying the utility scores (execution time, slowdown, and energy consumption) for each task, and applies data classification to define execution priorities, which are converted into execution ordering. Our simulation campaign demonstrated that KJS obtained promising results, improving the performance indicators when compared to individual policies. Moreover, KJS is easily extensible and can incorporate new information, policies and objectives into the training methodology. In summary, the contributions of this paper are three-fold:

- We created a database composed of multiple HPC DC and job statuses, obtained from scheduling real-world workloads with consolidated scheduling policies.

The rationale for creating this database is that although specialized literature offers several efficient scheduling policies in specific scenarios, there is no *silver bullet* scheduling policy.

In this sense, we composed a large knowledge dataset to guide the scheduling process.

- Among the existing ML-based techniques, we opted into using data regression and classification, based on supervised learning.

Essentially, two simple-explained techniques were combined for learning and composing the scheduling policy.

- Finally, results from the simulation campaign supported the identification of benefits and drawbacks of this approach.

The remainder of this paper is organized as follows: in Section 2 we define the research problem, motivation and present the related work. Section 3 details the dataset composition and the procedure for generating the scores, while Section 4 explains the data classification methodology used for composing the scheduling policy. The simulation campaign is presented in Section 5, and the final remarks are discussed in Section 6.

2 Problem definition and motivation

2.1 Job scheduling

Every running application in a HPC DC is to be interpreted as a job which composes a tree of child and parent tasks. The act of scheduling jobs is to determine an order of dealing with the demands placed by them. There is an arbitrarily long list of jobs and each must run for a determined, finite amount of time, using totally, or partially, the HPC DC infrastructure. The way a RJMS prioritizes tasks and distributes computing nodes is entirely up to its job scheduling policy, and the RJMS is tasked with maintaining cohesion in the HPC DC, while dealing with hundreds of tasks at the same time – and for this particular context, there is no preemption, that is, processes will run until they finish; no context switch [11].

For the purposes of this work, each task (t) has a set of characteristics, four of which are promptly used in the scheduling decision: requested time p_t , provided by the user; requested resources q_t for executing the task, time of arrival r_t , and waiting time w_t (how long the task is waiting to run). Once the task is scheduled, the starting time s_t is accounted, and after its execution, the total running time f_t is acquired.

Essentially, the RJMS schedules the jobs guided by a given objective function, which represents the HPC and the users' objectives. We focus on a scheduling scenario classified as open, with online submissions, which is in contrast but nevertheless matched by our proposed offline training methodology, the KJS; and among the multiple metrics that are available to represent users and HPC DC administrator perspectives [12], we focus on bounded slowdown (bsld) for representing users' jobs. The waiting time w_t of each task is accounted as $w_t = s_t - r_t$ and denotes how long a task waited before starting its execution. Following this line, the bounded slowdown for a given task is given by $bsld_t = \max\left\{\frac{w_t + p_t}{\max(p_t, \tau)}, 1\right\}$, where τ is a constant that is typically set in the order of 10 s, which prevents small tasks from having excessively large slowdowns [9, 12]. Finally, tasks postponed due to lack of DC resources remain queued to be processed in the next time

events (at this point the starvation phenomenon may be observed).

2.2 Well-known scheduling policies

The development of scheduling policies is constantly researched by the specialized literature to evolve the efficiency of HPC infrastructures. Each policy has its advantages (and eventually drawbacks) and optimizes specific scenarios, usually tailored to specific workloads. Recently, the use of well-known policies for composing large knowledge datasets is opening the research horizon for improving decision-making scenarios [5, 8–10]. Some traditionally used policies are summarized by Table 1, and were selected for composing the present work. These algorithms vary in time and space complexity, and some are, for a fact, a single criterion to separate one job from another. To start, Area First (SAF) allows jobs to run based on how wide their area (the product of time required and resources required) is; First Come First Served (FCFS) allows jobs to run based on order of arrival, first-come-first-served, along with the aggressive backfilling policy (EASY) that fills in idle spaces in a machine's timeline with smaller jobs that would not interfere with the pre-conceived schedule [9]; Shortest Job First (SJF) allows jobs to run based on their requested time; and lastly, functions F1 through F4 allow jobs to run based on their requested time, requested resources and time of arrival in a statistics-derived fashion [5].

2.3 Related work

The use of ML-based techniques to improve the overall performance of HPC schedulers or to propose new policies is a rich research field. In this sense, the specialized literature comprises works covering different learning techniques, and objectives. Initially, regarding the overall RJMS and infrastructure management, ML-guided

techniques are applied in multiple tasks. Casagrande et al. [13] shed light onto the grid computing sustainability and financial issues, seeing how the energy consumption increases almost linearly with resource utilization. An adaptable server shutdown strategy saves energy without a statistically relevant increase of shutdown events, and brings about energy-aware scheduling. Consolidated heuristics were used to master Internet congestion control algorithms [8] demonstrating the ability of data-driven decision-making in a traditional communication-related challenge. KJS could be applied after selecting the target HPC DC hosts. In fact, the technique proposed by Casagrande et al. prunes the search space, decreasing the number of comparisons required for defining the execution order.

The intersection between machine learning and parallel processing has been a key requirement for achieving our goal of a knowledge-driven new scheduling algorithm. In this context, the work [5] paved the road demonstrating how simulation and ML can be combined to obtain new decision-making policies. The authors introduced F1, F2, F3, and F4 policies, essentially proposing a methodology for composing new scheduling policies. We incorporated all four policies into the KJS's tutors, which improved the discussion by using simple data regression and classification techniques for ordering the tasks. Instead of composing new policies, KJS dynamically adapts the decision-making to the workload under analysis. In turn, Liu et al. [14] propose that a combination of deep reinforcement learning and parallelism would provide better adapted solutions for the Job Shop Scheduling Problem in real-life, dynamic systems with unpredicted shortcomings. Recently, [6] showed a similar approach to identify which heuristic should be applied in different HPC workloads. The authors used Actor-Critic Reinforcement Learning for defining the appropriate scheduling policy. This decision is taken before scheduling a subset of HPC jobs based on DC usage and users' perspectives. This work explores leveraging pre-trained data regression and classification models, rather than continuously generating a decision-making model from scratch. While pre-trained, these models are generated with a large-scale knowledge dataset encompassing various decisions, job statuses, job queues, and other relevant parameters.

2.4 Challenges and research opportunities

Although efficient in the scenarios in which they were originally proposed, existing policies vary in performance indicators used in other scenarios. For exemplifying, Figure 1 summarizes the bounded slowdown metric as histograms, demonstrating how the decision-making impacts on different workloads (the details on simulation campaign

Table 1 Task-based policies and their parameter formulas

Policy	Formula
FCFS	r_i
SPT	p_i
SQF	q_i
SAF	$p_i \cdot q_i$
F1	$\log_{10}(p_i) \cdot q_i + 8.70 \cdot 10^2 \cdot \log_{10}(r_i)$
F2	$\sqrt{p_i} \cdot q_i + 2.56 \cdot 10^4 \cdot \log_{10}(r_i)$
F3	$p_i \cdot q_i + 6.86 \cdot 10^6 \cdot \log_{10}(r_i)$
F4	$p_i \cdot \sqrt{q_i} + 5.30 \cdot 10^5 \cdot \log_{10}(r_i)$

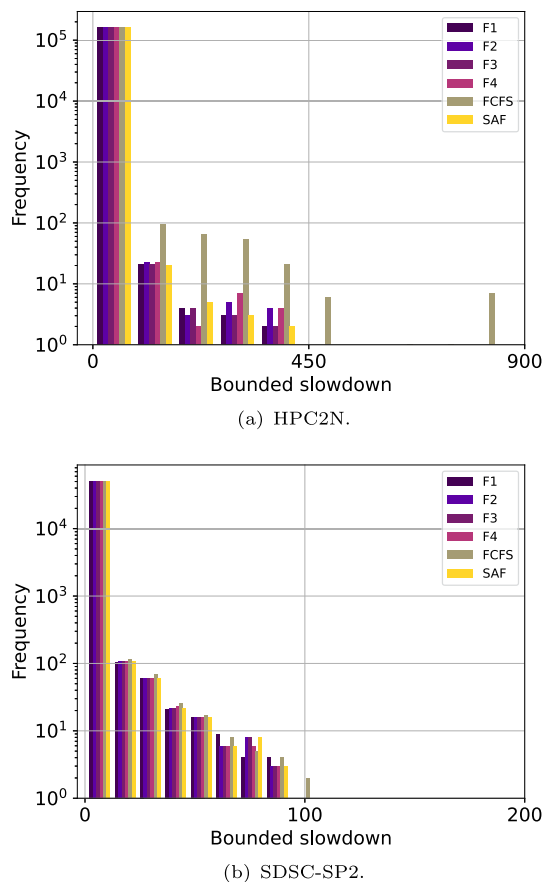


Fig. 1 Bounded slowdown for different workload databases and scheduling policies

are presented in Section 5). For instance, while SAF and some ML-generated policies were efficient in the Figure 1(a) scenario, the performance indicators decreased on Figure 1b, obtained from a distinct workload. In addition, a simple FCFS has competitive results for Figure 1b, while having largely increased the bounded slowdown in Figure 1(a). The example makes it clear that there isn't a silver-bullet scheduling policy, corroborating the opportunity to apply ML techniques aiming for improvement of such complex task. We claim that in addition to the human knowledge applied in the development of existing policies, the information resulting from the application of such policies to schedule real jobs constitutes fundamental data to improve scaling in general.

In this context, instead of proposing a one-size-fits-all scheduling policy, our work applies an ML-based methodology (polynomial regression and data classification) for ordering the jobs. It is worthwhile to mention that any heuristic and model previously proposed (Section 2.3) can be used in the training methodology. Figure 2 summarizes the main stages composing the KJS training methodology, which are individually discussed in the following sections.

3 Learning from HPC schedulers

The combination of requests, DC statuses, applications' workloads, and policies' decisions composes a large and powerful knowledge dataset. Deep analysis can indicate a policy's dominance for a given metric, as well as distinct conclusions for objectives based on dichotomies. Following the specialized literature (Section 2.3) we compose and analyze the dataset (Section 3.1), innovating by applying multiple perspectives, termed scores (Section 3.2).

3.1 Simulating HPC scenarios

To simulate a HPC scenario, a myriad of information in the form of parallel workload logs in Standard Workload Format [15] was fed into the Python API for the PyBatSim simulation framework.¹ Essentially, PyBatSim is a high-level Application Programming Interface (API) enabling the use of the robust Batsim [16] simulator in Python. Therefore, PyBatSim was selected to maintain the use of a single programming language for the entire project and simultaneously run the simulations in a consolidated tool. The result is that each entry in a workload was scheduled by each of the algorithms (from Section 2.2), composing the dataset. In addition to the jobs' descriptive attributes (from Section 2.1), a set of information obtained after applying the scheduling policies was selected. To summarize, each row of the resulting dataset is composed of: job's release date; walltime; job's requirements; the scheduling algorithm that took on this job; the number of idle machines in the datacenter; the number of computing machines in the datacenter; the waiting time; and the execution time. The DC-related attributes represent the status after performing the scheduling decision.

This step is illustrated by bullet 1 in Figure 2, demonstrating in a simplified manner with the cartesian product of scheduling policies, parallel workloads and DC platforms, such that the assignment of score values generates a plethora of data ready to be used for ML. In the context of our research, two distinct workloads, namely, SDSC-SP2 and HPC2N, were selected for the training of our ML models, demonstrating the execution of our methodology (the simulation campaign is detailed in Section 5).

3.2 Computing scores

Subsequent to the generation of the dataset representing every conceivable combination of schedulers and workloads, KJS computes scores of each scheduling decision (as depicted by bullet 2 on Figure 2). Each score represents a performance indicator for a given metric. Initially, we

¹ PyBatSim available at: <https://gitlab.inria.fr/batsim/pybatsim>.

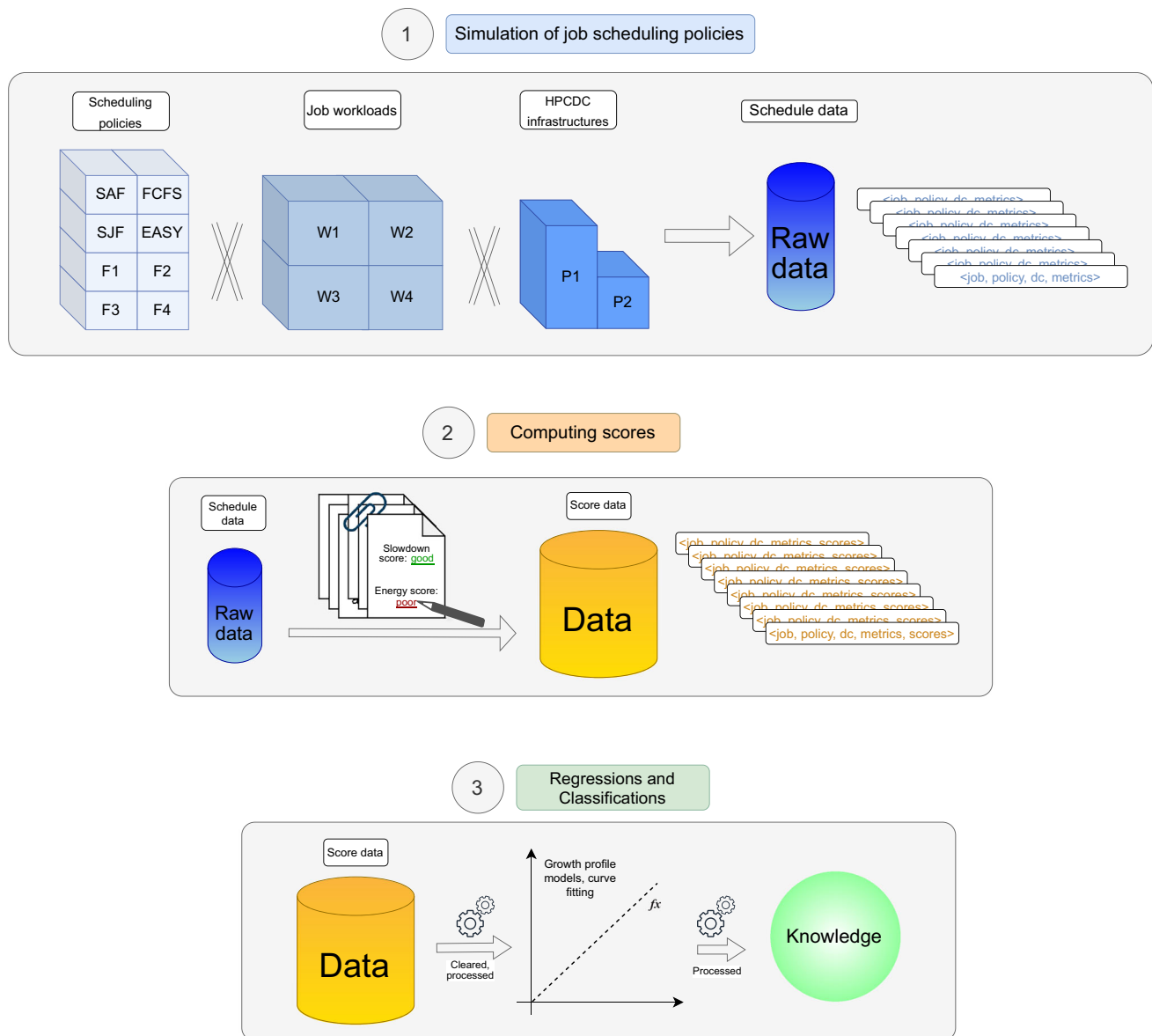


Fig. 2 The main stages composing the KJS training methodology

opted for using bounded slowdown, execution time, and energy consumption as the starting point. The rationale behind the scores is to quantify the positive/negative impact of the decision-making for each individual line in the dataset. These exhaustive calculated indicators are fundamental to understand the performance of consolidated scheduling policies when ordering jobs, respecting the HPC DC configurations.

The function used to assign a score for energy consumption places the wattage value for each job before the average energy consumption of the workload. Essentially, we account the impact on the HPC DC consumption when a new job is scheduled. This information is estimated from existing models for predicting the energy consumption

based on the running time and server details. The less energy it takes to schedule this job, the better a score for the job.

In turn, the function used to assign a slowdown score consists on evaluating the turnaround time and the execution time of a job, putting them against each other to assert the job's contribution to the overall fluidity of the workload. This score captures the impact on overall scheduling efficiency. When prioritizing a job, the scheduler might delay the execution of others waiting in the queue. This concept aligns directly with assigning priorities in multi-queue scheduling systems. In our work, we consider slowdown to enrich the knowledge dataset for our ML models. This allows them to learn from a wider range of

scheduling decisions, leading to improved task execution overall. The less a job slows down the workflow, the better a score this job will get. Last but not least, the function used to assign a score for the execution time simply considers the time taken to run the task or job after it was scheduled. This score reflects the accuracy of user-estimated execution times. Ideally, users should provide precise estimates. However, some users may choose the maximum allowed walltime when requesting resources, even if the actual execution time is shorter. This mismatch impacts scheduling efficiency.

The conceptualization of scores facilitates the portrayal of the distinctive attributes inherent to each job scheduling policy. To illustrate, if a given scheduling policy primarily takes into consideration the submission time of the jobs, the associated score would be computed by normalizing the job submission times to a standardized scale, ranging from 0 to 1. This normalization process ensures that all input parameters are represented on a consistent scale.

4 KJS implementation

The implementation details of all three main stages from Figure 2 are discussed in the following sections. Specifically, bullets 1 and 2 are presented in Section 4.1, while bullet 3 is detailed in Section 4.2.

4.1 Predicting the scores

Seeking to predict the score of new jobs before effectively scheduling them, we made use of polynomial regression. Regression is, as a whole, a manner of predicting continuous numerical values, based on previously recorded values, using statistics. This was implemented in the Python language with the help of the scikit-learn library. In summary, scikit-learn library offers an implementation of ordinary least squares method for a linear regression model. We used it with the default parameters. Preprocessing consisted of extracting a rather large, pseudorandom excerpt of the original score data; then converting it into appropriate formats, cleaning it of statistical outliers based on each sample's Local Outlier Factor with a number of relative neighbors equal to 200, and, finally, normalizing it.

After splitting the set of values of the composite x variable and the set of values of the single y variable into test and train sets, it was possible to fit a polynomial regression model to the data and extrapolate to a number of possible curves. These curves would represent the growth profile or variation of a particular metric in the dataset, as shows Figure 2, bullet 3. Comparison of predicted values and expected values involved the use of a metric called Root

Mean Squared Error (RMSE) that asserts the quality of the retrieved models (presented in Section 5). It is important that the variables used for predictions are readily known before the job is scheduled.

4.1.1 Predicting the average slowdown

Slowdown speaks of the job's performance in the ideal environment of a dedicated system, where the job has all of the resources to itself [11]. Such metric of central importance in the associated literature is no different in this research, but the authors used it in the form of average slowdown, which is taken from each job's contribution. To predict the average slowdown y in a set of parallel workloads, five attributes were used (composing x): the number of idle machines and conversely, the number of machines that shall process the job; the walltime, or time required for execution; the revenue, or machines made available for execution; and the waiting time.

4.1.2 Predicting the energy consumption

Energy consumption is a consequence of the job's processing. It is one of the less tricky attributes to predict, considering its proportional growth to the time and to the amount of resources requested by a job. This is calculated taking into account a value of wattage per state that is fed to Batsim through HPC DC platform configuration. It is taken in joules. To predict the energy consumption y in a set of parallel workloads, four attributes, exactly one less than average slowdown, were used: the number of idle machines; the number of machines that shall process the job; the walltime; and the revenue.

4.1.3 Predicting the execution time

Execution time is only really known after the particular job has run. This attribute is an estimate, and is taken into account based on the simulated reality's passing of time, in a job-centric view of the results of the scheduling. It is taken in seconds. To predict the execution time y in a set of parallel workloads, two attributes were used: the walltime and the revenue (the number of computing resources reserved for a given job).

4.2 Classifying and ordering jobs

Given our current ability to predict job scores, KJS can have the (positive or negative) impact of each task composing a queue. Specifically, at each time that the scheduler is urged to make a decision, KJS uses the predefined models to get scores regarding the users' and administrators' perspectives. KJS relies on the Gaussian Naive Bayes

classification technique [17] as it outperformed its counterparts in preliminary tests.

Upon the reception of a job by PyBatSim (or any real RJMS), KJS extracts relevant job attributes and employs the trained polynomial regressions to anticipate the scores. Subsequently, KJS standardizes these values in consideration of all delivered jobs that remain unscheduled. Following the normalization of values, these standardized metrics are processed. Leveraging this data, the KJS forecasts the queue position for each of the unscheduled jobs and then sorts them based on the queue position.

At this point, any predicted score, or a combination of multiple scores, can be used to classify the job queue. Essentially, KJS aims at optimizing a given objective function composed of representative scores, and it is worthwhile to mention that KJS can be easily extended to incorporate new scores and objective functions.

4.3 Prototype details and implementation

The prototype was implemented in Python version 3.10.12 supported by scikit-learn version 1.2.2. Simulators, models, and other tools were executed on a server equipped with 2 Intel(R) Xeon(R) Silver 4214 CPU @ 2.20GHz totaling 24 cores and 48 threads, with 128 GB RAM. Regarding the offline training process, Algorithms 1 and 2 summarize the main points. KJS uses offline training and once the model is ready for use, it can be connected with any RJMS or simulator (as performed in Section 5) to schedule on-line jobs.

Algorithm 1 KJS regression training process.

```

Input: A set of parallel workload traces  $\mathcal{W}$ 
Input: A set of tutor scheduling policies  $\mathcal{P}$ 
Input: A set of score functions  $\mathcal{S}$ 
Input: A HPC DC  $\mathcal{H}$ 
1  $pybat\text{sim} \leftarrow \text{PyBatSim}()$ ;
2  $database \leftarrow \emptyset$ ;
3 for  $w \in \mathcal{W}$  do
4   for  $p \in \mathcal{P}$  do
5      $schedule\_map \leftarrow$ 
        $pybat\text{sim}.schedule(jobs(w), p, \mathcal{H})$ ;
6     for  $s \in \mathcal{S}$  do
7       for  $t \in jobs(w)$  do
8          $score \leftarrow s.get(t, schedule\_map)$ ;
9          $database.append(t, schedule\_map, score)$ ;
10      end
11    end
12  end
13 end
14 for  $s \in \mathcal{S}$  do
15    $model \leftarrow polynomial\_regression(s, database)$ ;
16    $model.save()$ ;
17 end

```

The KJS regression training process is summarized by Algorithm 1. Initially, the parallel workload traces, the tutor policies, the score functions, and the current DC status are informed. As discussed, the simulation process is supported by PyBatSim (line 1) for composing a knowledge database (line 2). The score for each individual task (lines 8 and 9) is calculated and appended to the database, which is used for training the regression models (lines 14 to 17). In turn, Algorithm 2 depicts how a Gaussian Naive Bayes technique is trained to infer the scheduling. Instead of sorting the queue of tasks that must be scheduled, KJS combines all scores (lines 4 to 7) for improving the knowledge database. This information is used as input for training, and once saved is given as input for classifying (appointing priorities) tasks. Once the database is completed, KJS trains a data classification model (line 10) splitting the dataset in 70% for training and 30% for validation. The model is saved and can be incorporated to any RJMS.

Algorithm 2 KJS scheduling training process.

```

Input: A parallel workload trace  $w$ 
Input: A set of scores functions  $\mathcal{S}$ 
Input: A HPC DC  $\mathcal{H}$ 
1  $regression\_models \leftarrow load\_models(\mathcal{S})$ ;
2  $database \leftarrow \emptyset$ ;
3 for  $t \in jobs(w)$  do
4   for  $s \in \mathcal{S}$  do
5      $score \leftarrow models(s).predict(t, \mathcal{H})$ ;
6      $database.append(t, score, s)$ ;
7   end
8 end
9  $database.sort(scores)$ ;
10  $gnb.train()$ ;
11  $gnb.save\_model()$ ;

```

5 Simulations

For discussing the simulation protocol and results, initially the comparative scenarios (Section 5.1) are presented, followed by metrics and parameters (Section 5.2). Finally, Section 5.3 summarizes the results and discussions.

5.1 Scenarios and tutors

Following the bullet 1 from Figure 2, all eight policies listed in Section 2.2 and summarized in Table 1 were selected as tutors, for generating the knowledge dataset. Each tutor scheduled multiple traces from Parallel Workloads Archive [15] atop two HPC DCs, composed of 64 and 32 homogeneous nodes (different from the original one used in the trace files). At this point it must be noted that

we used separate datasets for training the models and for conducting the comparisons. The polynomial regressions and data classification training (bullets 2 and 3 from Figure 2, respectively) were performed with SDSC-SP2 and HPC2N datasets (also used for composing the example in Figure 1), while the comparisons are based on SDSC-BLUE workload scheduled atop a 64-node homogeneous DC. Workloads SDSC-SP2, HPC2N, and SDSC-BLUE are composed of 59715, 202871, and 243306 jobs, respectively. In terms of distinct internal users' profiles, they have 437, 257, and 468 applications submitted, respectively. It is worthwhile to mention that the original workloads describe the job queue and HPC infrastructure status after the decision-making process. Moreover, each workload's file contains data from distinct policies (among the listed in Table 1), placing the jobs atop distinct HPC infrastructures. In our work, we are ignoring the original scheduling decision and using only the jobs' data (mainly resources demand and arrival times). However, one may note that implicitly, the original decision-making scheduler can influence the arrival time, since the users are aware of the queue status before submitting their jobs. This scenario is beneficial for composing a heterogeneous dataset for training the KJS models.

Figure 3 details the original workload's data. We extracted the arrival time, number of requested processors, and required runtime for composing the KJS dataset. Specifically for the arrival time (Figure 3(a)), the workloads have different submission rates, as well as distinct total makespan. In turn, the requested runtime (Figure 3c) demonstrates a distinct, however similar profile among all workloads. Finally, the workload used for comparing KJS with the counterparts and tutor algorithms is composed of jobs requiring more computing resources.

5.2 Metrics and parameters

Following the discussion from Sections 2.1 and 4.1, we selected Root Mean Squared Error (RMSE) for discussing

the regression models, and bounded slowdown (bsld), waiting time and turnaround time for representing users' and administrators' perspectives when scheduling jobs. RMSE was acquired from the scikit-learn library, while schedule-related metrics were collected directly from the PyBatSim log files.

The results selected for discussing the polynomial regressions are restricted to degrees 1 (linear) up to 4, and are presented as Cumulative Distribution Function (CDF). In turn, bounded slowdown is discussed using histograms, organized by the workload characteristics. Four histograms were generated, with y indicating the frequency of data occurrence in x . These histograms are categorized according to processing time of tasks, segmenting the p_t values into specific ranges. Let $p_t^{max} = 1$, four intervals are defined: (i) $p_t \leq 0.25$; (ii) $0.25 < p_t \leq 0.5$; (iii) $0.5 < p_t \leq 0.75$; and (iv) $0.75 < p_t \leq 1$. This categorization approach allows for a more detailed analysis of the distribution of data with respect to different p_t ranges, which represent distinct workloads. In addition, a weekly consolidation is also presented, as well as a summarization of maximum, minimum, and mean values for schedule-related metrics. Finally, the scheduled-related metrics are presented only for FCFS, SAF, and F1, which outperformed the counterpart tutors.

5.3 Results

5.3.1 Predicting scores

The polynomial regression for each score (execution time, bounded slowdown, and energy consumption) are presented in Figure 4. Upon close examination of the results, it is possible to visualize the CDF that each polynomial regression model yields.

Normalization had been part of our dataset preprocessing, which does not mean values predicted cannot exceed or come short of the original zero to one range, as they would proportionally correspond to values close to - if only

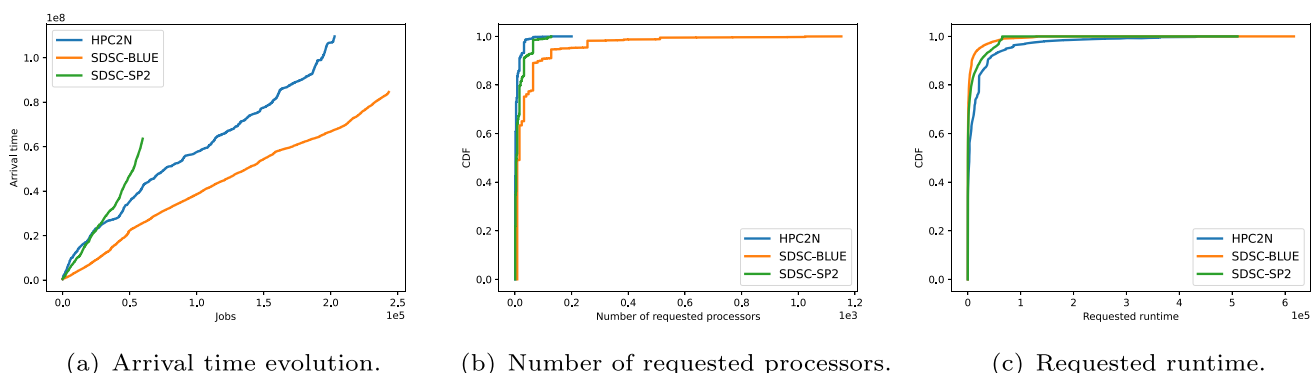


Fig. 3 Details on original workload's data. KJS was trained with SDSC-SP2 and HPC2N, and analyzed with SDSC-BLUE

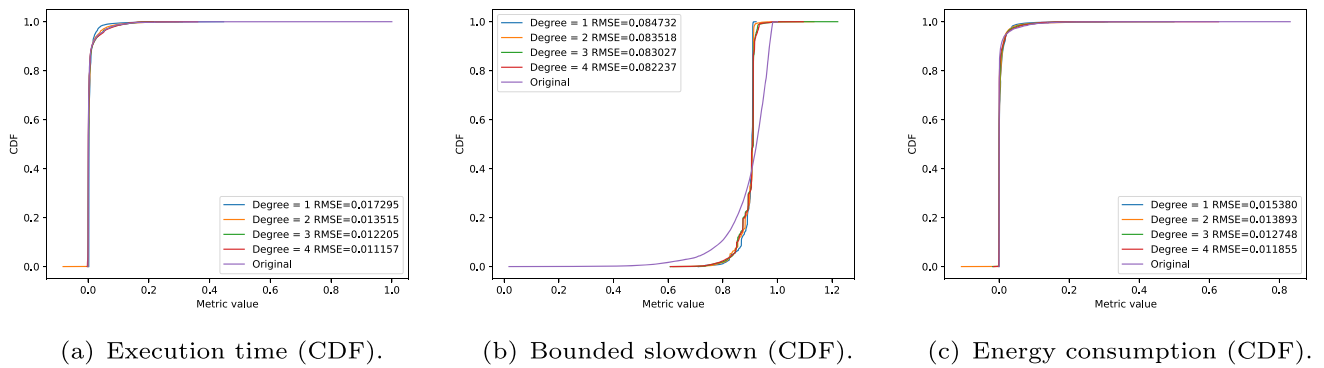


Fig. 4 Polynomial regression for scheduling parameters

a little offset from - the real values for the variable. Thus, we can see that now the x axis corresponds to the possible values for the predicted variable mentioned in each subtitle, and the new y axis is a percentage of values to lay in the specific range that x entails. In Figure 4, the purple curve should be interpreted as the real distribution wherein the range of each value, that is, where in a scale of 0 to a maximum each value should reside, is represented for the original dataset, and all of the other curves are each model's attempt at prediction. Especially in the energy consumption case, prediction of values was nearly perfect, meaning all of the retrieved regression models performed excellently in their affair. In every case, the fourth degree polynomial was chosen as the best fit one, which shows a possible tendency of a decreasing RMSE with the increase of the regression polynomial degree. We deemed it appropriate to settle for degree four considering the small increase in the quality of prediction, versus the computational cost of operations, with higher degree polynomials.

5.3.2 Scheduling new jobs

Figure 5 presents the bounded slowdown values for FCFS, SAF and KJS after scheduling the SDSC-BLUE jobs atop an homogeneous 64-nodes DC, using the regression models from Section 5.3.1. Results are grouped by the workload characteristic in terms of processing time (p_r , Figures 5(a) to 5(b)), followed by macro analysis in Figures 5e and 5f. In addition, Table 2 includes the F1 ML-guided model, summarizing the minimum, maximum, and mean values for all policies.

For small-scale tasks (Figures 5(a) and 5(b)), the histograms demonstrate that KJS reduced the number of tasks suffering from slowdown, even when compared with SAF. As the $bsld$ values increase, the histograms indicate a trend where fewer jobs are scheduled with KJS. This observation suggests that, particularly in scenarios with higher $bsld$ values, KJS is effective in decreasing slowdowns.

By increasing the task processing requirements (Figures 5c and 5d), KJS kept the performance indicators. In short, KJS combined the knowledge from the tutor scheduling policies to compose new scheduling scenarios, decreasing the weekly average cumulative bounded slowdown (Figure 5f). In this way, it is possible to conclude that the model assimilated the patterns of the tutor algorithms during learning, resulting in a combined scheduling policy that, for the most part, decreases the occurrence of high $bsld$ values.

In addition, Table 2 summarizes the metrics (normalized). For bounded slowdown, KJS kept competitive mean values, obtaining the slower maximum. In turn, although the maximum waiting time for a single task was observed with KJS even comparing to FCFS approach, it obtained the same mean values. These metrics indicated that KJS can amalgamate the knowledge from all tutors, keeping the mean values competitive for all metrics.

5.4 Extending KJS for different scenarios

The methodology for creating KJS considered a modular evolution and extension for different scenarios. Using Figure 2 to conduct the discussion, a new scenario can replace individual stages (or the whole scenario). Initially, the knowledge database can be extended adding new tutors, workloads, and HPC infrastructures. However, it is worthwhile to note that this procedure is an extension; the current dataset version already comprises information from multiple data sources. Following the rationale, a HPC DC administrator can add new scores representing the different perspectives (e.g., the network load). In this case, once the scores have been calculated, a regression-based model must be trained, to predict values that will be used for performing the decision-making. Finally, a future work could replace the data classification approach for a distinct model, taking advantage of the precedent stages of KJS methodology.

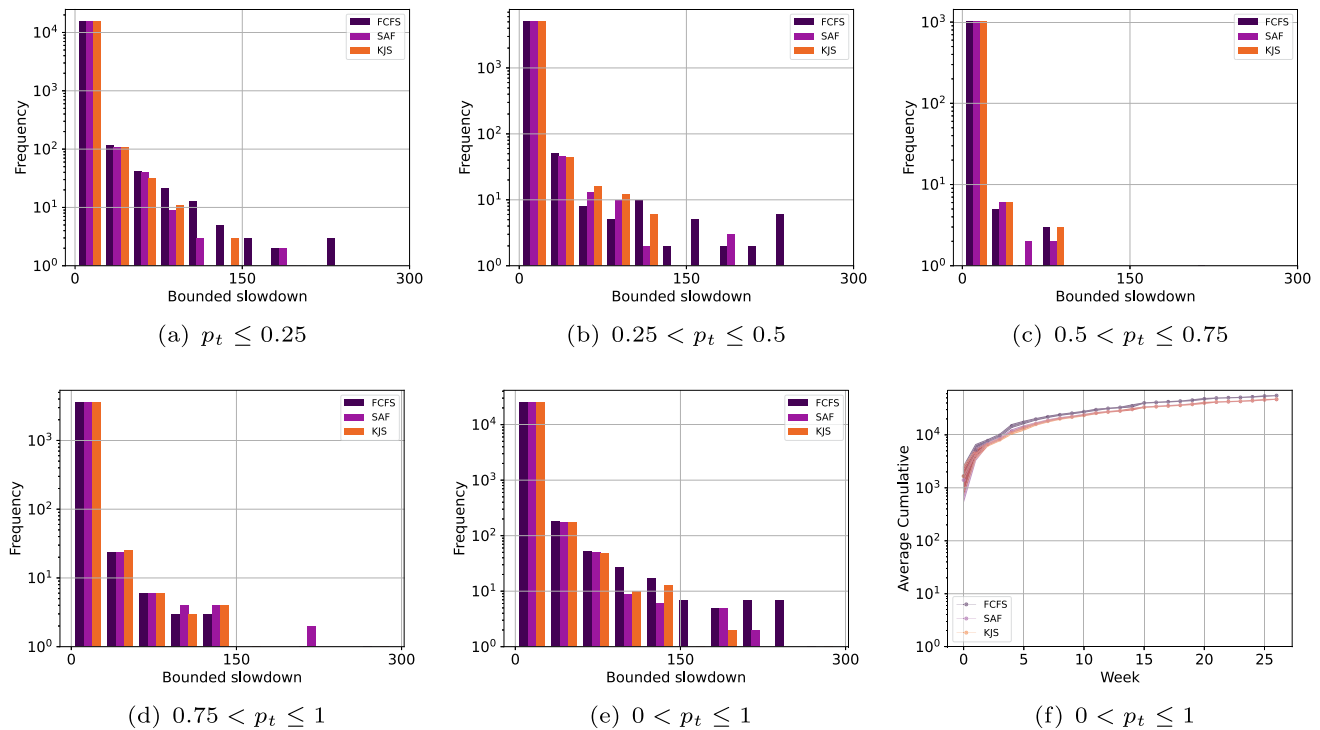


Fig. 5 Summary results of bounded slowdown for SDSC-BLUE

Table 2 Summary of metrics and scheduling policies

	KJS	FCFS	SAF	F1
Max bounded slowdown	0.6	1.0	0.7	0.7
Mean bounded slowdown	0.6	1.0	0.6	0.6
Max waiting time	1.0	0.8	0.9	0.9
Mean waiting time	0.8	1.0	0.8	0.8
Mean turnaround time	0.9	1.0	0.9	0.9

6 Conclusion

High-Performance Computing (HPC) infrastructures are shared by multiple users, which submit applications' jobs with distinct requirements in terms of computing power, reservation time, and scale. In this context, a Resources and Jobs Management System (RJMS) relies on efficient scheduling policies to optimize the infrastructure performance and usage indicators. Given such importance, the specialized literature comprehends multiple proposals based on different techniques and eventually tailored for specific scenarios and workloads. In this work, we considered this situation as an opportunity to propose a supervised learning based methodology for improving the performance of scheduling policies. In summary, we presented Knowledge-based Job Scheduler (KJS), based on data regression and classification for defining the

appropriated task execution ordering. Our simulation campaign demonstrated that KJS obtained promising results, improving the performance indicators when compared to individual policies. Moreover, KJS is easily extensible and can incorporate new information, policies and objectives into the training methodology. As future work, we plan to train and evaluate KJS with different workloads and arrival distribution functions, as well as analyze the performance from new scheduling tutors.

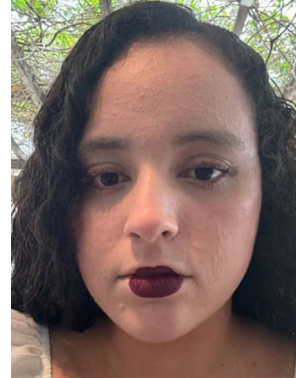
Acknowledgements This work was funded by the National Council for Scientific and Technological Development (CNPq), the Santa Catarina State Research and Innovation Support Foundation (FAPESC), Santa Catarina State University (UDESC), and developed at Laboratory of Parallel and Distributed Processing (LabP2D).

References

1. Dongarra, J., Luszczek, P.: *TOP500*. Boston, MA: Springer US, pp. 2055–2057. [Online]. Available: https://doi.org/10.1007/978-0-387-09766-4_157 (2011)
2. Reed, D., Gannon, D., Dongarra, J.: “Reinventing high performance computing: Challenges and opportunities,” (2022)
3. Abbasloo, S., Yen, C.-Y., Chao, H. J.: “Classic meets modern: a pragmatic learning-based congestion control for the internet,” in *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication*, ser. SIGCOMM '20. New York, NY, USA: Association for Computing Machinery, p. 632–647. [Online]. Available: <https://doi.org/10.1145/3387514.3405892> (2020)

4. Yen, C.-Y., Abbasloo, S., Chao, H. J.: “Computers can learn from the heuristic designs and master internet congestion control,” in *Proceedings of the ACM SIGCOMM 2023 Conference*, ser. ACM SIGCOMM '23. New York, NY, USA: Association for Computing Machinery, p. 255–274. [Online]. Available: <https://doi.org/10.1145/3603269.3604838> (2023)
5. Carastan-Santos, D., de Camargo, R. Y.: “Obtaining dynamic scheduling policies with simulation and machine learning,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '17. New York, NY, USA: Association for Computing Machinery, (2017). [Online]. Available: <https://doi.org/10.1145/3126908.3126955>
6. Koslovski, G. P., Pereira, K., Albuquerque, P. R.: “Dag-based workflows scheduling using actor-critic deep reinforcement learning,” *Future Generation Computer Systems*, vol. 150, pp. 354–363, (2024). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X23003485>
7. Li, J., Zhang, X., Wei, J., Ji, Z., Wei, Z.: “Garsched: Generative adversarial deep reinforcement learning task scheduling optimization for large-scale high performance computing systems,” *Future Generation Computer Systems*, vol. 135, pp. 259–269, (2022). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X22001613>
8. Yen, C.-Y., Abbasloo, S., Chao, H. J.: “Computers can learn from the heuristic designs and master internet congestion control,” in *Proceedings of the ACM SIGCOMM 2023 Conference*, (2023), pp. 255–274
9. Carastan-Santos, D., De Camargo, R.Y., Trystram, D., Zrigui, S.: “One can only gain by replacing easy backfilling: A simple scheduling policies case study,” in: 19th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID) **2019**, 1–10 (2019)
10. Blenk, A., Kalmbach, P., Kellerer, W., Schmid, S.: “O’zapft is: Tap your network algorithm’s big data!” in *Proceedings of the Workshop on Big Data Analytics and Machine Learning for Data Communication Networks*, (2017), pp. 19–24
11. Feitelson, D. G., Rudolph, L., Schwiegelshohn, U., Sevcik, K. C., Wong, P.: “Theory and practice in parallel job scheduling,” in *Job Scheduling Strategies for Parallel Processing*, D. G. Feitelson and L. Rudolph, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 8, 27 (1997)
12. Feitelson, D. G., Rudolph, L.: “Metrics and benchmarking for parallel job scheduling,” in *Workshop on Job Scheduling Strategies for Parallel Processing*. Springer, pp. 1–24 (1998)
13. Casagrande, L., Koslovski, G., P. Miers, C.C., M.A., Gonzalez, N.: “Don’t hurry be green: scheduling servers shutdown in grid computing with deep reinforcement learning,” in *International Journal of Grid and Utility Computing*. Inderscience Publishers, (2022)
14. Liu, C.-L., Chang, C.-C., Tseng, C.-J.: Actor-critic deep reinforcement learning for solving job shop scheduling problems. *IEEE Access* **8**, 71 752–71 762 (2020)
15. Feitelson, D.G., Tsafir, D., Krakov, D.: Experience with using the parallel workloads archive. *Journal of Parallel and Distributed Computing* **74**(10), 2967–2982 (2014)
16. Dutot, P.-F., Mercier, M., Poquet, M., Richard, O.: “Batsim: a realistic language-independent resources and jobs management systems simulator,” in *Job Scheduling Strategies for Parallel Processing: 19th and 20th International Workshops, JSSPP: Hyderabad, India, May 26, 2015 and JSSPP 2016, Chicago, IL, USA, May 27, 2016, Revised Selected Papers 19*. Springer **2017**, 178–197 (2015)
17. Ontivero-Ortega, M., Lage-Castellanos, A., Valente, G., Goebel, R., Valdes-Sosa, M.: “Fast gaussian naïve bayes for searchlight classification analysis,” *NeuroImage*, vol. 163, pp. 471–479, (2017). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1053811917307371>

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Guilherme Diel is an undergraduate student at Santa Catarina State University (UDESC) in Joinville/SC, Brazil. Currently, he holds a scholarship at the Laboratory of Parallel and Distributed Processing (labP2D) at UDESC, where he has mainly been involved in addressing the Scheduling problem within the context of High-Performance Computing data centers.



Ana Kraus is an undergraduate student at Santa Catarina State University (UDESC) in Joinville/SC, Brazil. She is currently a scholar for the Laboratory of Parallel and Distributed Processing at UDESC and has worked with the Job Shop Scheduling problem in the context of High Performance Computing data centers.



Guilherme Piêgas Koslovski is professor of Computer Networks, Parallel Programming, and Distributed Systems at Santa Catarina State University (UDESC) in Joinville/SC - Brazil. He received his doctorate from École Normale Supérieure at Lyon, France, the master’s degree from Federal University of Santa Maria (UFSM), Brazil, and his bachelor’s degree from the UFSM, Brazil, in Computer Science. Currently, his research is related to high performance computing, virtual infrastructures, scheduling, software defined networks, and virtualization of computational and communication resources. He is the coordinator of LabP2D (Laboratory of Parallel and Distributed Processing) at UDESC.